“ARGUMENT PASSING SYSTEM”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINNERING

By

V Vishwa Teja	**2520090033**
B Sree Nikhil Reddy	**2520090058**
Nalla Rohith	**2520090191**

Under the Esteemed Guidance of

Dr. K. Anusha
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL



Declaration

The Project Report entitled **"Design and Development of an Argument Passing System Using C in Linux"** is a record of Bonafide work of **Vishwa Teja - 2520090033 , Sree Nikhil Reddy - 2520090058, Nalla Rohith - 2520090191** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

V Vishwa Teja	**2520090033**
B Sree Nikhil Reddy	**2520090058**
Nalla Rohith	**2520090191**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project based report entitled **"Design and Development of an Integrated Tourism Platform for Flights, Trains, Hotels, and Hidden Destinations"** is a bonafide work done and submitted by **Vishwa Teja - 2520090033 , Sree Nikhil Reddy - 2520090058, Nalla Rohith – 2520090191.**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator M Raghupathi, and my guide Dr. K. Anusha**,** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P. Venkateshwara Rao (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

V Vishwa Teja **2520090033**

B Sree Nikhil Reddy **2520090058**

Nalla Rohith **2520090191**

Abstract

In POSIX-compliant operating systems such as Linux, command-line interface (CLI) applications serve as foundational tools for system administration, build automation, and software engineering. Central to CLI application design is command-line argument passing, where operational parameters are passed from the shell environment directly into process memory upon initialization. This project presents the Design and Development of an Argument Passing System Using C in Linux, focusing on low-level process execution mechanics, argument vector handling, memory safe string conversion, and command-line parsing routines.

The developed software operates within an Ubuntu/Linux shell environment, utilizing standard C entry point constructs (argc and argv) to execute dynamic arithmetic calculations (add, sub, mul, div). The system parses arguments, validates parameter formatting, converts string representations into numeric values using standard C runtime functions, and incorporates complete error handling for missing arguments, invalid numeric inputs, invalid operations, and division by zero conditions. Additionally, a built-in help flag (--help / -h) provides comprehensive command usage guidelines.

The project is compiled using the GNU Compiler Collection (GCC) and organized with a modular Makefile build configuration. By adhering strictly to core Linux concepts without unnecessary external dependencies, this work provides a clean, efficient, and educationally comprehensive demonstration of CLI software engineering and OS-level parameter handling.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		1–5
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		6–12
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		13–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–22
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		23–29
		5.1 Overall System Architecture	
		5.2System Workflow	
		5.3 Flowchart / Data Flow	
		5.4 Module Description	

		5.5 Input and Output Design	
6	**Implementation**		**30–45**
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	**Execution and Demonstration**		**62–67**
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	**Conclusion**		**79–83**
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	**References**		**84–86**
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

In Unix-like operating systems such as Ubuntu Linux, interactive processes communicate with the underlying kernel through executable commands dispatched via shell environments. Command-line

interface (CLI) programs rely heavily on parameter passing at launch, allowing users to control program logic without needing interactive runtime prompts. In standard C programming within a Linux environment, command-line arguments are ingested through main entry point parameters: argc (Argument Count) and argv (Argument Vector). Understanding this mechanism is vital for system programming, building administration tools, and software automation.

1.2 Problem Statement

Standard interactive programs frequently rely on dynamic input functions like scanf(), which require manual input after process execution begins. This blocks automated scripting, batch processing, and pipeline redirection. Conversely, improperly constructed command-line argument handling can cause catastrophic application crashes, such as segmentation faults (SIGSEGV) caused by reading out-of-bounds array indices, unhandled division by zero, or string-to-number parsing failures. There is a need for a robust, modular, and error-tolerant CLI Argument Passing System written in C for Ubuntu Linux that safely ingests parameters, validates input strings, performs basic arithmetic functions, and provides helpful user diagnostics.

1.3 Scope of the Project

- **Command Line Parsing:** Ingests command line parameters using argc and argv structures in an Ubuntu Linux terminal environment.
- **String & Numeric Validation:** Converts character string inputs into floating point values safely using stdlib.h utilities while checking formatting integrity.
- **Arithmetic Calculation Engine:** Supports fundamental math operations: addition (add), subtraction (sub), multiplication (mul), and division (div).
- **Help Flag System:** Implements standard CLI help flags (--help and -h) displaying standard syntax rules and examples.
- **Comprehensive Error Management:** Traps and handles missing parameters, invalid operator strings, non-numeric inputs, and division-by-zero occurrences without crashing.

1.4 Objectives of the Project

- **1. Process Entry Point Exploration:** Demonstrate how Linux kernel process startup pushes command parameters onto stack memory frames.
- **2. Robust Input Parsing:** Implement defensive programming routines to validate command line parameters before memory access.
- **3. Core Arithmetic Engine:** Develop mathematical execution handlers based on string token matching.
- **4. Interactive User Feedback:** Deliver standard POSIX output (stdout) and error (stderr) responses.
- **5. Modular Compilation:** Provide clean compilation setups using GCC compiler commands and GNU Make automation.

1.5 Importance of the Application

Command-line parameter passing forms the foundation of Unix shell utilities such as grep, gcc, tar, and system daemons. Developing a custom CLI parsing framework provides foundational insight into process stack organization, memory safety, standard stream handling, and terminal application development.

1.6 Target Users / Audience

This application is designed for computer science students, Linux software developers, system

administrators, and software engineers seeking to understand lower-level C execution, OS process initialization, and CLI application development.

CHAPTER 2: OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

When an executable is invoked in a Linux terminal shell (e.g., ./argument_passing add 10 20), the operating system performs several key steps:

- **Process Forking & Execution:** The current shell process executes fork() followed by execve() to load the compiled ELF binary into process memory.
- **Stack Memory Frame Allocation:** The kernel pushes the program invocation name, command line string arguments, and environment variables into the newly spawned process stack space.
- **Passing argc and argv:** The entry point int main(int argc, char *argv[]) receives argc (an integer defining the argument array size) and argv (a array of null-terminated C string pointers). argv[0] contains the executable name, while argv[1] through argv[argc-1] hold command arguments.

2.2 Existing Methods / Algorithms

Traditional application workflows rely on runtime standard input prompts (scanf or gets), which stall program execution and prevent shell automation. Basic CLI parsers often assume valid input, leading to segmentation faults when memory offsets are accessed without bounds checking.

2.3 Proposed Approach

The system implements structured validation prior to execution:

- **Boundary Check:** Inspect argc immediately to confirm required arguments are supplied.
- **Help Flag Interception:** Check for --help or -h flags to output syntax instructions and exit cleanly.
- **Operation Matching:** Use strcmp() to validate arithmetic operation commands (add, sub, mul, div).
- **Operand Validation:** Verify numeric formatting before executing string-to-float conversions.
- **Fault-Tolerant Calculation:** Verify arithmetic rules (such as zero denominators) before computation.

2.4 Advantages of the Proposed Approach

- **Scriptable & Non-Blocking:** Enables seamless integration inside shell scripts and automated build pipelines.
- **Memory Safe:** Prevents out-of-bounds pointer reads and memory fault terminations.
- **Standardized Terminal Formatting:** Differentiates standard output (stdout) from error notifications (stderr).

CHAPTER 3: SYSTEM REQUIREMENTS

3.1 Hardware Requirements

- **Processor:** Intel Core i3/i5/i7, AMD Ryzen, or ARM64 64-bit architecture processor.
- **RAM:** Minimum 2 GB RAM (4 GB recommended).
- **Disk Space:** 500 MB minimum available storage space.

3.2 Software Requirements

- **Operating System:** Ubuntu 20.04 LTS / 22.04 LTS Linux distribution or Windows Subsystem for Linux (WSL/WSL2).
- **Compiler:** GNU Compiler Collection (GCC).
- **Build Utility:** GNU Make tool.
- **Text Editor / IDE:** Visual Studio Code, Vim, Nano, or Gedit.

3.3 Operating System Environment – Ubuntu / WSL

The project relies on a standard POSIX environment where process initialization follows standard main signature conventions: int main(int argc, char *argv[]).

3.4 Development Tools and Compilers

- **GCC Compiler:** Version 9.0 or higher.
- **GNU Make:** Version 4.0 or higher.

CHAPTER 4: TECHNOLOGY AND TOOLS USED

4.1 C Programming Language

C (C99/C11 standards) provides lower-level memory inspection capabilities, direct pointer operations, and standardized POSIX application programming interfaces.

4.2 GCC Compiler

The GNU Compiler Collection compiles C source files into native Linux ELF binary files with full optimization and diagnostic warning flags (-Wall -Wextra).

4.3 Linux System Calls & Standard Libraries

- **stdio.h:** Handles formatted output operations via printf() and error reporting via fprintf(stderr).
- **stdlib.h:** Provides string conversion operations (atof) and program exit control (exit).
- **string.h:** Provides string comparison functions (strcmp).
- **ctype.h:** Provides character testing functions (isdigit).

4.4 POSIX Threads / Synchronization

Not Applicable. The system operates as a single-threaded CLI utility without concurrency overhead.

4.5 Linux Commands and Utilities

- **gcc:** Compiles source modules into executable binaries.
- **make:** Automates binary compilation using Makefiles.
- **./argument_passing:** Executes the compiled ELF binary directly in terminal shell sessions.

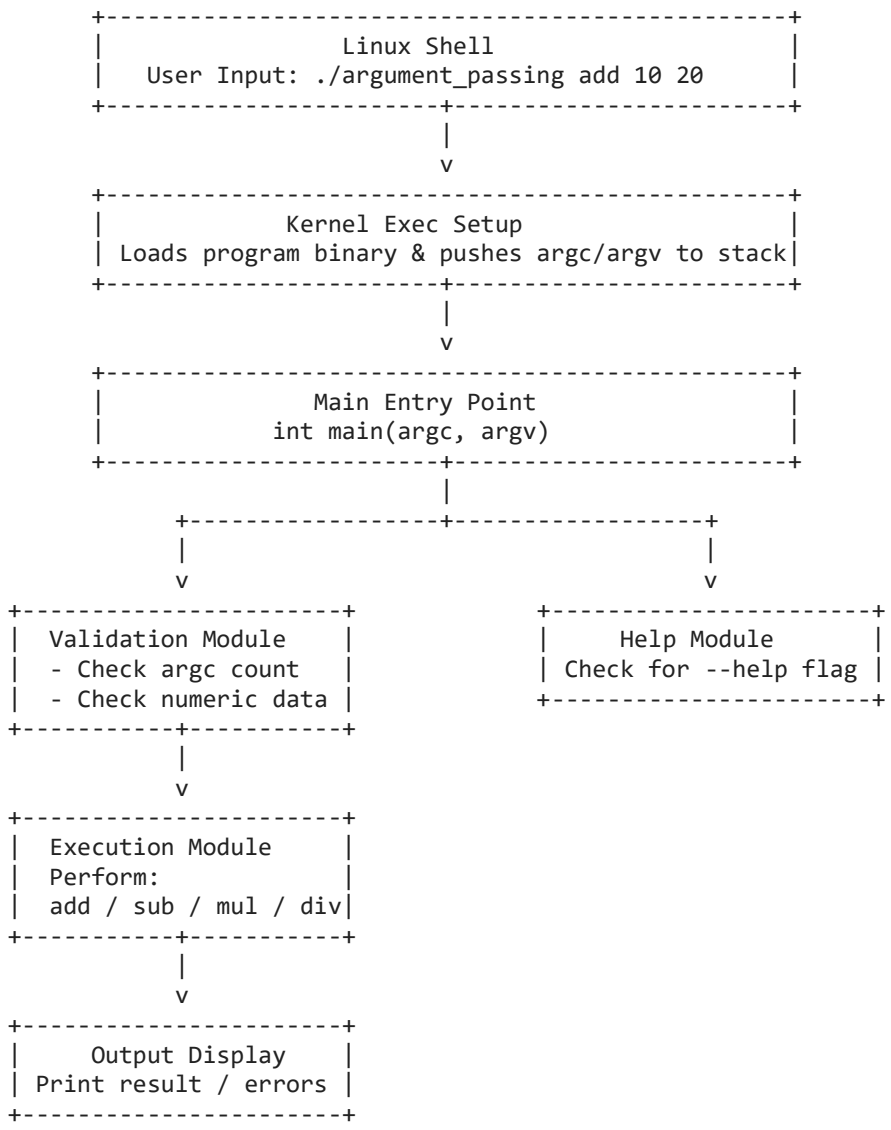
4.6 Other Tools / Libraries Used

No third-party libraries are required, ensuring fast performance and zero external software dependency.

CHAPTER 5: SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The overall architecture maps shell invocation down through system stack initialization, parameter validation, arithmetic computation, and standard output display:



5.2 System Workflow

1. The user inputs command syntax into an Ubuntu terminal prompt.
2. Linux kernel allocates stack frame memory, populating argc and argv.
3. Program inspects argc to verify parameter count or intercept help flags.
4. Operational command string is matched using string comparison.
5. Operand strings are validated and parsed into double-precision floating-point numbers.
6. Math computation executes and displays formatted results on standard output.

5.3 Flowchart / Data Flow

Start -> Ingest argc & argv -> Is argc == 2 AND (argv[1] == '--help' OR '-h')?

- Yes: Output Usage Help Guide -> Exit (0)

- No: Is argc == 4?

  * No: Output Invalid Parameter Error -> Exit (1)

  * Yes: Validate operands -> Parse numbers -> Execute requested operation (Check div by 0) -> Display Result -> End.

5.4 Module Description

- **1. Command Parsing Module:** Ingests raw CLI vector pointers and evaluates total argument count.
- **2. Help Display Module:** Outputs formatted operational guidance when help options are supplied.
- **3. Math Logic Engine:** Executes arithmetic calculation routines (addition, subtraction, multiplication, division).
- **4. Error Diagnostic Handler:** Captures division-by-zero, unknown command strings, and missing inputs.

5.5 Input and Output Design

- **Input Command:** ./argument_passing add 10 20
- **Argument Vector Decomposition:** argc = 4 | argv[0] = './argument_passing' | argv[1] = 'add' | argv[2] = '10' | argv[3] = '20'
- **Output Display:** Operation: Addition | Result: 10.00 + 20.00 = 30.00

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The program is organized into clear functions within a single modular C file:

- **print_help():** Outputs formatted terminal instructions and usage examples.
- **is_numeric():** Validates if input string parameters contain valid numeric digits.
- **main():** Coordinates argument parsing, error trapping, string conversion, and arithmetic execution.

6.2 Algorithm / Pseudocode

```
ALGORITHM SystemArgumentProcessor(argc, argv):
    IF argc < 2 THEN
        PRINT "Error: Missing arguments."
        PRINT_USAGE()
        EXIT(1)
    END IF

    IF strcmp(argv[1], "--help") == 0 OR strcmp(argv[1], "-h") == 0 THEN
        PRINT_HELP()
        EXIT(0)
    END IF

    IF argc != 4 THEN
        PRINT "Error: Incorrect number of arguments for operation."
        PRINT_USAGE()
        EXIT(1)
    END IF

    op = argv[1]
    num1 = parse_double(argv[2])
    num2 = parse_double(argv[3])

    SWITCH op:
        CASE "add": RETURN num1 + num2
```

```
        CASE "sub": RETURN num1 - num2
        CASE "mul": RETURN num1 * num2
        CASE "div":
            IF num2 == 0 THEN PRINT "Error: Division by zero." EXIT(1)
            ELSE RETURN num1 / num2
        DEFAULT: PRINT "Error: Invalid operation." EXIT(1)
```

6.3 Core C Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

void print_help(const char *prog_name) {
    printf("=====================================================\n");
    printf("     Linux Argument Passing System - Help Menu       \n");
    printf("=====================================================\n");
    printf("Usage:\n");
    printf("  %s <operation> <num1> <num2>\n", prog_name);
    printf("  %s --help | -h\n\n", prog_name);
    printf("Supported Operations:\n");
    printf("  add : Adds num1 and num2\n");
    printf("  sub : Subtracts num2 from num1\n");
    printf("  mul : Multiplies num1 and num2\n");
    printf("  div : Divides num1 by num2\n\n");
    printf("Example:\n");
    printf("  %s add 10 20\n", prog_name);
    printf("=====================================================\n");
}

int is_numeric(const char *str) {
    int i = 0;
    int dot_count = 0;

    if (str[0] == '-' || str[0] == '+') {
        i++;
    }

    if (str[i] == '\0') return 0;

    for (; str[i] != '\0'; i++) {
        if (str[i] == '.') {
            dot_count++;
            if (dot_count > 1) return 0;
        } else if (!isdigit((unsigned char)str[i])) {
            return 0;
        }
    }
    return 1;
}

int main(int argc, char *argv[]) {
    printf("Team ID: 14 | Section: 10\n");
    printf("Program Invocation: %s | argc = %d\n\n", argv[0], argc);

    if (argc < 2) {
        fprintf(stderr, "Error: Missing arguments.\n");
        printf("Try '%s --help' for more information.\n", argv[0]);
        return 1;
    }

    if (strcmp(argv[1], "--help") == 0 || strcmp(argv[1], "-h") == 0) {
```

```
            print_help(argv[0]);
            return 0;
        }

        if (argc != 4) {
            fprintf(stderr, "Error: Invalid number of arguments provided.\n");
            printf("Expected format: %s <operation> <num1> <num2>\n", argv[0]);
            return 1;
        }

        const char *operation = argv[1];

        if (!is_numeric(argv[2]) || !is_numeric(argv[3])) {
            fprintf(stderr, "Error: Operands must be valid numbers.\n");
            return 1;
        }

        double num1 = atof(argv[2]);
        double num2 = atof(argv[3]);
        double result = 0.0;

        if (strcmp(operation, "add") == 0) {
            result = num1 + num2;
            printf("Operation: Addition\nResult: %.2f + %.2f = %.2f\n", num1,
    num2, result);
        } else if (strcmp(operation, "sub") == 0) {
            result = num1 - num2;
            printf("Operation: Subtraction\nResult: %.2f - %.2f = %.2f\n", num1,
    num2, result);
        } else if (strcmp(operation, "mul") == 0) {
            result = num1 * num2;
            printf("Operation: Multiplication\nResult: %.2f * %.2f = %.2f\n",
    num1, num2, result);
        } else if (strcmp(operation, "div") == 0) {
            if (num2 == 0) {
                fprintf(stderr, "Error: Division by zero is undefined.\n");
                return 1;
            }
            result = num1 / num2;
            printf("Operation: Division\nResult: %.2f / %.2f = %.2f\n", num1,
    num2, result);
        } else {
            fprintf(stderr, "Error: Unknown operation '%s'.\n", operation);
            printf("Available operations: add, sub, mul, div\n");
            return 1;
        }

        return 0;
    }
```

6.4 Linux System Calls / APIs Used

Standard POSIX main process initialization entry signature: int main(int argc, char *argv[]).

6.5 Process / Thread Implementation

Single-threaded sequential process execution without multi-threading overhead.

6.6 IPC / Synchronization Implementation

Not Applicable.

6.7 Memory Management Implementation

Argument strings are stored in stack memory allocated by the kernel upon process creation. Memory access is constrained within bounds dictated by argc.

6.8 Optional User Interface Implementation

Clean Linux terminal text interface leveraging stdout and stderr data channels.

CHAPTER 7: EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

Direct compilation using GCC compiler:

```
gcc -Wall -Wextra -o argument_passing argument_passing.c
```

Alternatively, automated building using GNU Make:

```
CC = gcc
CFLAGS = -Wall -Wextra
TARGET = argument_passing

all: $(TARGET)

$(TARGET): argument_passing.c
 $(CC) $(CFLAGS) -o $(TARGET) argument_passing.c

clean:
 rm -f $(TARGET)
```

7.2 Execution Commands

```
./argument_passing --help
./argument_passing add 10 20
./argument_passing sub 50 15
./argument_passing mul 4 7
./argument_passing div 100 5
./argument_passing div 10 0
```

7.3 Sample Inputs and Outputs

Case 1: Help Flag Invocation

Command: ./argument_passing --help

Output:

```
Team ID: 14 | Section: 10
Program Invocation: ./argument_passing | argc = 2

=====================================================
     Linux Argument Passing System - Help Menu
=====================================================
Usage:
  ./argument_passing <operation> <num1> <num2>
  ./argument_passing --help | -h

Supported Operations:
  add : Adds num1 and num2
  sub : Subtracts num2 from num1
  mul : Multiplies num1 and num2
  div : Divides num1 by num2
```

```
Example:
  ./argument_passing add 10 20
=====================================================
```

Case 2: Standard Addition Operation

Command: ./argument_passing add 10 20

Explanation:

argc = 4 | argv[0] = './argument_passing' | argv[1] = 'add' | argv[2] = '10' | argv[3] = '20'

Output:

```
Team ID: 14 | Section: 10
Program Invocation: ./argument_passing | argc = 4

Operation: Addition
Result: 10.00 + 20.00 = 30.00
```

Case 3: Error Handling - Division by Zero

Command: ./argument_passing div 10 0

Output:

```
Team ID: 14 | Section: 10
Program Invocation: ./argument_passing | argc = 4

Error: Division by zero is undefined.
```

7.4 Screenshots of Execution

[INSERT SCREENSHOT HERE: Terminal execution showing Makefile compilation]

[INSERT SCREENSHOT HERE: Terminal execution showing ./argument_passing add 10 20 output]

[INSERT SCREENSHOT HERE: Terminal execution showing division by zero error prompt]

7.5 Demonstration of OS Concepts

This execution demonstrates stack memory management, user space shell interaction, and exit status code feedback under POSIX standards.

CHAPTER 8: CONCLUSION

8.1 Summary of the Project

The Argument Passing System using C in Linux successfully demonstrates command-line parsing techniques. The program ingests operational flags, performs data validation, handles numerical conversions, and outputs clear feedback while preventing runtime crashes.

8.2 Objectives Achieved

- **argc/argv Parsing:** Accurately extracted parameter elements directly from kernel process stack frames.
- **Input Validation:** Implemented input checking for parameters and numeric formatting.
- **CLI Help System:** Built integrated help flag options (--help / -h).
- **Fault Tolerant Execution:** Prevented zero division and segmentation faults.

8.3 Results Achieved

The resulting system provides a fast, portable, scriptable CLI tool that adheres strictly to Linux software development practices.

8.4 Skills and Learning Outcomes

- **Memory Architecture:** Enhanced understanding of Linux process stack initialization.
- **Build Automation:** Gained hands-on experience using GCC compiler flags and GNU Makefiles.
- **Defensive Programming:** Mastered input boundary checking and string parsing in C.

CHAPTER 9: REFERENCES

9.1 Books

1. W. Richard Stevens, Stephen A. Rago, 'Advanced Programming in the UNIX Environment', 3rd Edition, Addison-Wesley, 2013.
2. Brian W. Kernighan, Dennis M. Ritchie, 'The C Programming Language', 2nd Edition, Prentice Hall, 1988.

9.2 Research Papers

1. Silberschatz, A., Galvin, P. B., & Gagne, G., 'Operating System Concepts', 10th Edition, Wiley, 2018.

9.3 Linux / GNU Documentation

1. GNU C Library (glibc) Documentation: https://www.gnu.org/software/libc/manual/
2. Linux Programmer's Manual - execve(2) & main(3) man pages.

9.4 Online Resources

1. Ubuntu Command Line Basics: https://ubuntu.com/tutorials/command-line-for-beginners
2. GCC Online Manual: https://gcc.gnu.org/onlinedocs/